

Módulo 5: Tipado estático opcional y calidad

Contenidos clave

Type hints y typing avanzado (Union, Literal, TypedDict, Protocol):

- Type hints: son anotaciones que indican qué tipo de dato espera una variable, parámetro o función, permiten que herramientas como mypy o Copyright detecten posibles errores antes de ejecutar el programa.
- Union: indica que un valor puede ser de uno u otro tipo.
`def buscar_usuario(id: Union[int, str]) -> str:`
`return f"Usuario: {id}"`
entonces:
`buscar_usuario(123) # puede ser un int`
`buscar_usuario("ABC123") # puede ser un str`
- Literal: restringe un valor a opciones específicas. "solo puede tener estos valores en concreto".
- TypedDict: permite definir qué claves y tipos debería tener un diccionario. Estas herramientas de tipado saben qué estructura debería tener.
- Protocol: permite definir qué comportamientos debe tener un objeto, sin exigir que pertenezca a una clase específica.

mypy/pyright; límites del tipado dinámico:

- mypy/pyright: son herramientas que realizan verificación estática de tipos. Analizan el código sin necesidad de ejecutarlo ("se esperaba int pero se recibió str"), pueden advertir. mypy: verificador estático de tipos muy utilizado en Python. Copyright: verificador estático, desarrollado por Microsoft y utilizado también por herramientas como Pylance en VS Code.

Los type hints describen los tipos y mypy/Pyright los comprueban.

Python es un lenguaje de tipado dinámico, lo que significa que una variable puede cambiar de tipo, esto proporciona flexibilidad, por ejemplo, si se espera un str y aparece un int, el error aparecerá durante la ejecución, con type hints un verificador estático puede detectar el problema antes.

PEP 8 aplicado con ruff/black/isort; PEP 20 como guía de diseño: PEP 8 es la guía oficial de estilo de Python, establece recomendaciones para mantener el código legible y consistente.

- Ruff: herramienta rápida para analizar código Python y detectar problemas de estilo y calidad.
- Black: formateador automático. Su función principal es tomar código con diferentes estilos y darle un formato consistente.
- isort: organiza automáticamente los import de Python.

PEP 20. Serie de principios que sirven como guía de diseño. La idea es preferir soluciones simples, claras, legibles y explícitas, sobre soluciones innecesariamente complicadas.

Pre-commit y checks en CI: Pre-commit permite ejecutar automáticamente ciertas verificaciones antes de hacer un commit en Git. Sigue el flujo: Escribir código -> git commit -> Pre-commit -> Ruff Black isort mypy -> ¿Todo correcto? -> Si -> Commit

\-> No -> Se detiene, así se evita que código con problemas llegue al repositorio.

Checks en CI (Continuous Integration / Integración Continua) significa que cada vez que alguien sube cambios al repositorio, un sistema automático puede ejecutar verificaciones. Ejemplo: Desarrollador -> Git push -> GitHub / GitLab -> CI -> Ruff -> mypy -> Tests -> ¿Todo Correcto? Si hay un error de tipos o una regla de calidad incumplida, el pipeline puede marcar el cambio como fallido.

Pre-commit verifica antes del commit en el entorno local; CI verifica después de subir los cambios, normalmente en un entorno compartido.

Objetivos

- *Anotar tipos y verificar estáticamente.*

Dentro del código hay variedad de type hints, por poner un ejemplo:

```
12      def enviar(self, mensaje: str, destinatario: str) -> bool:
```

Mensaje debe ser str, destinatario debe ser str y la función devuelve bool.

Se restringen los valores permitidos con Literal en:

```
5      EstadoNotificacion = Literal["pendiente", "completado", "fallido"]
6      Prioridad = Literal["baja", "media", "alta"]
```

Si hubiera un valor diferente a los establecidos como “urgente” en lugar de “alta” en prioridad, sería detectado por mypy o Pyright como un valor que no pertenece al Literal definido.

Instalación de mypy:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry add --group dev mypy
Using version ^2.3.1 for mypy

Updating dependencies
Resolving dependencies... (0.6s)

Package operations: 3 installs, 0 updates, 0 removals

- Installing ast-serialize (0.11.2)
- Installing librt (0.15.0)
- Installing mypy (2.3.1)

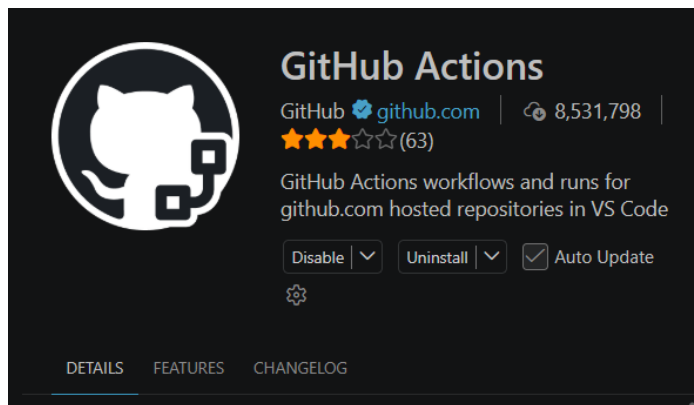
Writing lock file
```

- *Hacer cumplir PEP 8 y documentar excepciones.*

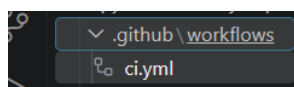
Ruff revisa estilo y calidad. Black mantiene el formato.

- *Integrar linters/formatters en CI.*

Instalación de la extensión GitHub Actions en VS Code.



CI corre en el servidor de GitHub Actions después de que se sube el código (push). Para configurarlo se crean las carpetas y el archivo ci.yml para configurarlo:



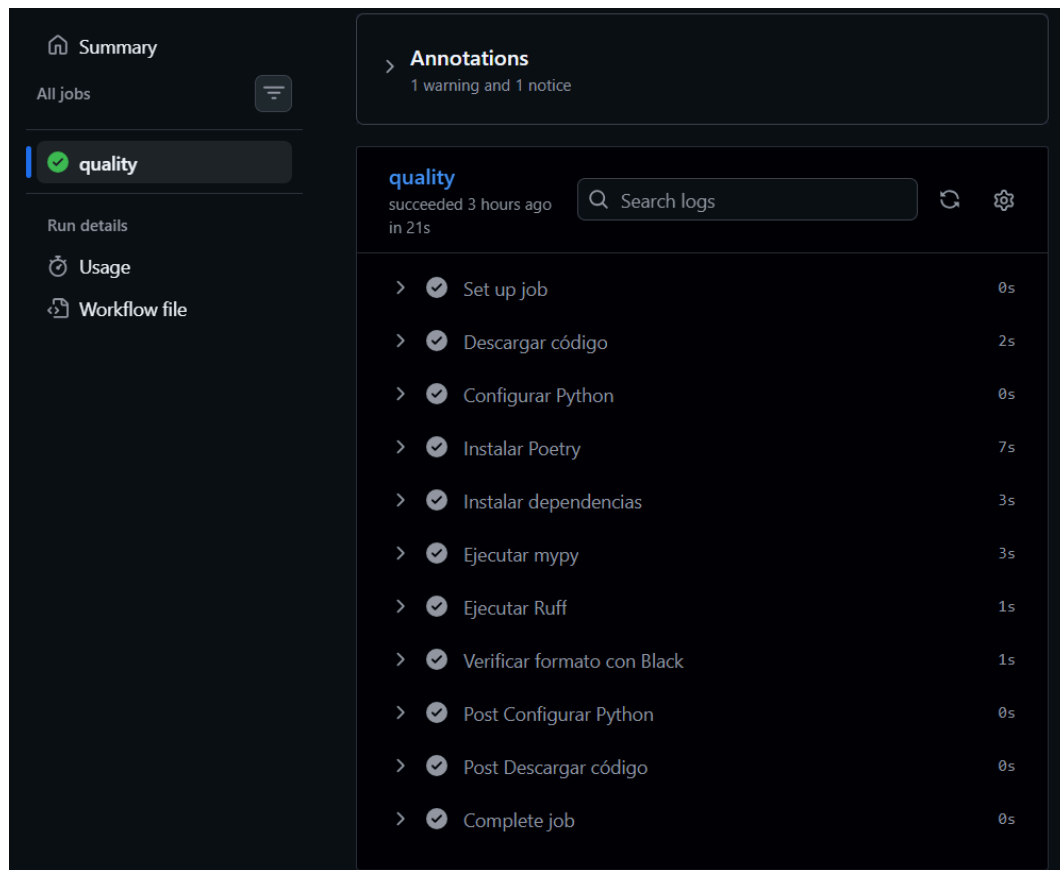
Se guarda y se sube:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git commit -m "configuración de CI para calidad de código"
[WARNING] Unstaged files detected.
[INFO] Stashing unstaged files to C:\Users\Quiryat\.cache\pre-commit\patch1789759574-9584.
black.....(no files to check)Skipped
isort.....(no files to check)Skipped
ruff.....(no files to check)Skipped
[INFO] Restored changes from C:\Users\Quiryat\.cache\pre-commit\patch1789759574-9584.
[feature/day-05-typing 9e764f8] configuración de CI para calidad de código
1 file changed, 35 insertions(+)
create mode 100644 .github/workflows/ci.yml
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git push
fatal: The current branch feature/day-05-typing has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/day-05-typing

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
```

Después en GitHub verificamos que todo esté correcto en la pestaña Actions:



The screenshot shows the GitHub Actions interface for a workflow named 'quality'. The workflow is in a 'succeeded' state, having completed 3 hours ago in 21 seconds. The left sidebar shows the workflow file and run details. The main area displays a list of 11 steps, each with a checkmark icon and a duration:

Step	Duration
Set up job	0s
Descargar código	2s
Configurar Python	0s
Instalar Poetry	7s
Instalar dependencias	3s
Ejecutar mypy	3s
Ejecutar Ruff	1s
Verificar formato con Black	1s
Post Configurar Python	0s
Post Descargar código	0s
Complete job	0s

A partir de ahora GitHub Actions ejecutará las comprobaciones automáticamente después del push.

Cuando se hace un push GitHub detecta el cambio haciendo que CI se ejecute automáticamente, instala las dependencias necesarias y corre los checks necesarios (mypy, Ruff, Black), si alguna herramienta encuentra un problema, el CI queda como fallido

Laboratorio

- Anotar tipos en el código previo, ejecutar mypy y ruff; configurar pre-commit.

Notación de tipos cubierta. Sequence y type hints indican los tipos, además de la información que espera la función.

```
37 def notificar_usuarios(  
38     canal: CanalNotificacion,  
39     destinatarios: Sequence[str],  
40     mensaje: str,  
41     prioridad: Prioridad | None = None,  
42 ) -> dict[str, EstadoNotificacion]:
```

Ejecución de mypy:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run mypy labs/day0  
5_typing/type_hints_advanced.py  
Success: no issues found in 1 source file
```

Ejecución de Ruff:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run ruff check .
All checks passed!
```

Pre-commit configurado previamente:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git commit -m "fix(day04)
: corregir tipado de payload_raw y model_validate para mypy / Agregado de init.py y con
figuración de explicit_package_bases para mypy / formateo de código y corrección de linter
s para ci"
black.....Passed
isort.....Passed
ruff.....Passed
[feature/day-05-typing dc30c46] fix(day04): corregir tipado de payload_raw y model_vali
date para mypy / Agregado de init.py y configuración de explicit_package_bases para myp
y / formateo de código y corrección de linters para ci
11 files changed, 386 insertions(+), 5 deletions(-)
create mode 100644 labs/Evidencias/__init__.py
create mode 100644 labs/__init__.py
create mode 100644 labs/day01_tooling/__init__.py
create mode 100644 labs/day02_control_flow/__init__.py
create mode 100644 labs/day03_pythonic/__init__.py
create mode 100644 labs/day04_datamodels/__init__.py
create mode 100644 labs/day05_typing/__init__.py
create mode 100644 labs/day05_typing/type_hints_advanced.py
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git branch
feature/day-01-environment
feature/day-02-control-flow
feature/day-03-pythonic
feature/day-04-datamodels
* feature/day-05-typing
main
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> git push
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 12 threads
```

Sistema de notificaciones tipado. Tiene la capacidad de enviar un mismo mensaje por diferentes canales (Email o SMS), usando Protocol para que la función principal no dependa de una implementación específica.

Se recibe una lista de usuarios y un mensaje. y lo envía mediante un servicio de Email o SMS. Además, utiliza tipado estático para controlar qué valores y estructuras son válidos.

Literal. Restringe valores: Esto ayuda a evitar valores incorrectos y permite que mypy los detecte.

```
5 EstadoNotificacion = Literal["pendiente", "completado", "fallido"]
6 Prioridad = Literal["baja", "media", "alta"]
```

Protocol. Define qué debe saber hacer un canal. Por lo que tanto ServicioEmail como ServicioSMS pueden ser válidos y utilizarse. @runtime_checkable comprueba el protocolo en ejecución.

```
10 @runtime_checkable
11 class CanalNotificacion(Protocol):
12     def enviar(self, mensaje: str, destinatario: str) -> bool:
13         """Cualquier clase que implemente este método satisface el protocolo."""
14         ...
```

Al final con:

```
# Verificación en tiempo de ejecución con @runtime_checkable
print(
    f"\n¿ServicioEmail cumple CanalNotificacion?: {isinstance(email_service, CanalNotificacion)}"
)
```

Se comprueba que ServicioEmail cumple con la estructura requerida por CanalNotificacion.